

Tecnatura Superior en Desarrollo de Software

## Anexo 05f — `IModelValidatorHub<T>`

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |  
Ciclo Lectivo 2026

### ¿Qué es un ModelValidatorHub?

**`IModelValidatorHub<T>`**Puerto de validación por  
modelo

Capa: Application

Un **ModelValidatorHub** es una clase cuya única responsabilidad es orquestar todos los Guards que corresponden a un modelo específico — generalmente un DTO de entrada — y devolver la lista consolidada de errores encontrados.

La interfaz `IModelValidatorHub<TModel>` define el contrato: dado un modelo de tipo `TModel`, producir una colección de `ValidationError`. Cada UseCase recibe el hub por inyección de dependencias y delega en él toda la validación de precondiciones — sin conocer los detalles de qué Guards se usan ni cómo se combinan.

**Claves:**

- Un hub por tipo de request
- Agrupa Guards de un DTO completo
- El UseCase solo llama a `Validate()`
- Implementa un puerto — es testeable
- Vive en Application

### Problema que resuelve:

Sin un hub, el UseCase contiene directamente todas las llamadas a `Guard.Ag` `ainst(...)` para cada campo del DTO. A medida que el DTO crece, el UseCase acumula responsabilidades de validación que no le corresponden — mezcla orquestación de negocio con verificación de precondiciones técnicas.

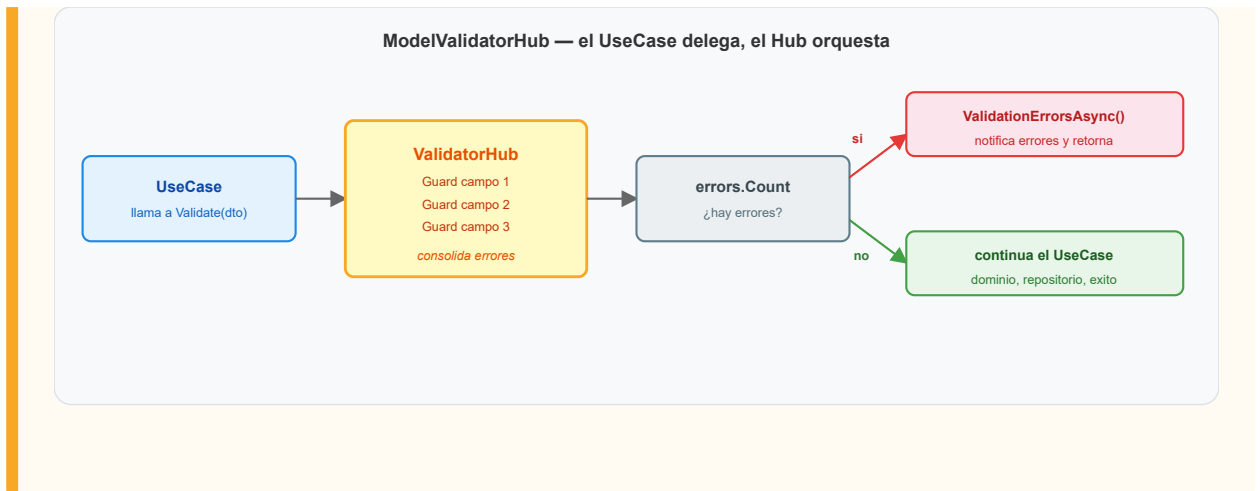
**El hub extrae toda esa lógica a una clase dedicada**, dejando al UseCase con una sola línea de validación.

### Analogía del mundo real

Imaginá el proceso de habilitación de un local comercial. Para obtener la habilitación, el dueño debe pasar por varias ventanillas: Bomberos, Bromatología, Habilitaciones, Planos. Cada ventanilla verifica una condición distinta.

En lugar de que el dueño sepa exactamente a qué ventanilla ir y en qué orden, existe un **gestor** que conoce todos los pasos, coordina las verificaciones y entrega al final una lista unificada de observaciones. El dueño solo habla con el gestor — no con cada ventanilla por separado.

El **ModelValidatorHub** es ese gestor: el UseCase le entrega el DTO y recibe la lista de todos los errores encontrados — sin saber qué Guards se ejecutaron internamente ni en qué orden.



## Características esenciales

### La interfaz — el contrato

`IModelValidatorHub<TModel>` es una interfaz genérica con un único método. El parámetro genérico `TModel` indica el tipo del modelo que se va a validar — generalmente el DTO de entrada del UseCase. El método devuelve todos los errores encontrados; si la colección está vacía, el modelo es válido.

```

1 // contrato genérico — un método, una responsabilidad
2 public interface IModelValidatorHub<TModel>
3 {
4     // valida el modelo y devuelve todos los errores encontrados
5     IEnumerable<ValidationError> Validate(TModel model);
6 }
  
```

### ¿Por qué IEnumerable y no List?

`IEnumerable<ValidationError>` es la abstracción más amplia para una secuencia de errores. El llamador puede convertirlo a `List` si necesita verificar `Count`, o simplemente iterar. La interfaz no impone una implementación concreta de colección — el hub decide internamente cómo acumula los errores.

## La implementación concreta

Cada request tiene su propio hub concreto que implementa la interfaz. El hub solo contiene llamadas a `Guard.Against()` — una por cada campo que necesita precondition técnica. Al final combina todos los errores y los devuelve como una sola secuencia.

```
1 // hub concreto para RegisterBookRequest
2 public class RegisterBookValidatorHub
3     : IModelValidatorHub<RegisterBookRequest>
4 {
5     public IEnumerable<ValidationError> Validate(RegisterBookRequest request)
6     {
7         // un Guard por cada campo que necesita precondition técnica
8         var g1 = Guard.Against(request.Title, "Title")
9             .NotNullOrEmpty().MaxLength(200);
10
11         var g2 = Guard.Against(request.Author, "Author")
12             .NotNullOrEmpty().MinLength(2).MaxLength(100);
13
14         var g3 = Guard.Against(request.ISBN, "ISBN")
15             .NotNullOrEmpty().MinLength(10).MaxLength(13);
16
17         // combina todos los errores en una sola secuencia y la devuelve
18         return g1.Errors
19             .Concat(g2.Errors)
20             .Concat(g3.Errors);
21     }
22 }
```

## El UseCase con hub — antes y después

El impacto en el UseCase es inmediato: toda la lógica de validación desaparece del método `ExecuteAsync` y queda reemplazada por una sola llamada a `_validator.Validate(request)`.

**SIN HUB — Guards dispersos en el UseCase**

El UseCase sabe exactamente qué Guards aplicar y cómo combinarlos — responsabilidad que no le corresponde.

```
1 public async Task ExecuteAsync(RegisterBookRequest request)
2 {
3     // UseCase conoce todos los Guards — acoplamiento fuerte
4     var g1 = Guard.Against(request.Title, "Title").NotNullOrEmpty().MaxLength(200);
5     var g2 = Guard.Against(request.Author, "Author").NotNullOrEmpty().MinLength(2);
6     var g3 = Guard.Against(request.ISBN, "ISBN").NotNullOrEmpty();
7     var errors = g1.Errors.Concat(g2.Errors).Concat(g3.Errors).ToList();
8     if (errors.Count > 0) { await _outputPort.ValidationErrorsAsync(errors);
9     return; }
10    // ... resto del UseCase
11 }
```

**CON HUB — UseCase limpio**

El UseCase delega la validación completa en una línea — no sabe qué Guards se aplican internamente.

```
1 public async Task ExecuteAsync(RegisterBookRequest request)
2 {
3     // una sola línea — el hub sabe qué validar
4     var errors = _validator.Validate(request).ToList();
5     if (errors.Count > 0)
6     {
7         await _outputPort.ValidationErrorsAsync(errors);
8         return;
9     }
10    // ... resto del UseCase sin ruido de validación
11 }
```

## ¿Por qué se usa?

Sin el hub, cada vez que el DTO de entrada cambia — se agrega un campo, se modifica una regla de longitud, se elimina una validación — el UseCase debe modificarse. Esto viola el **principio de responsabilidad única**: el UseCase debería cambiar solo cuando cambia la lógica de negocio, no cuando cambia la forma de validar la entrada.

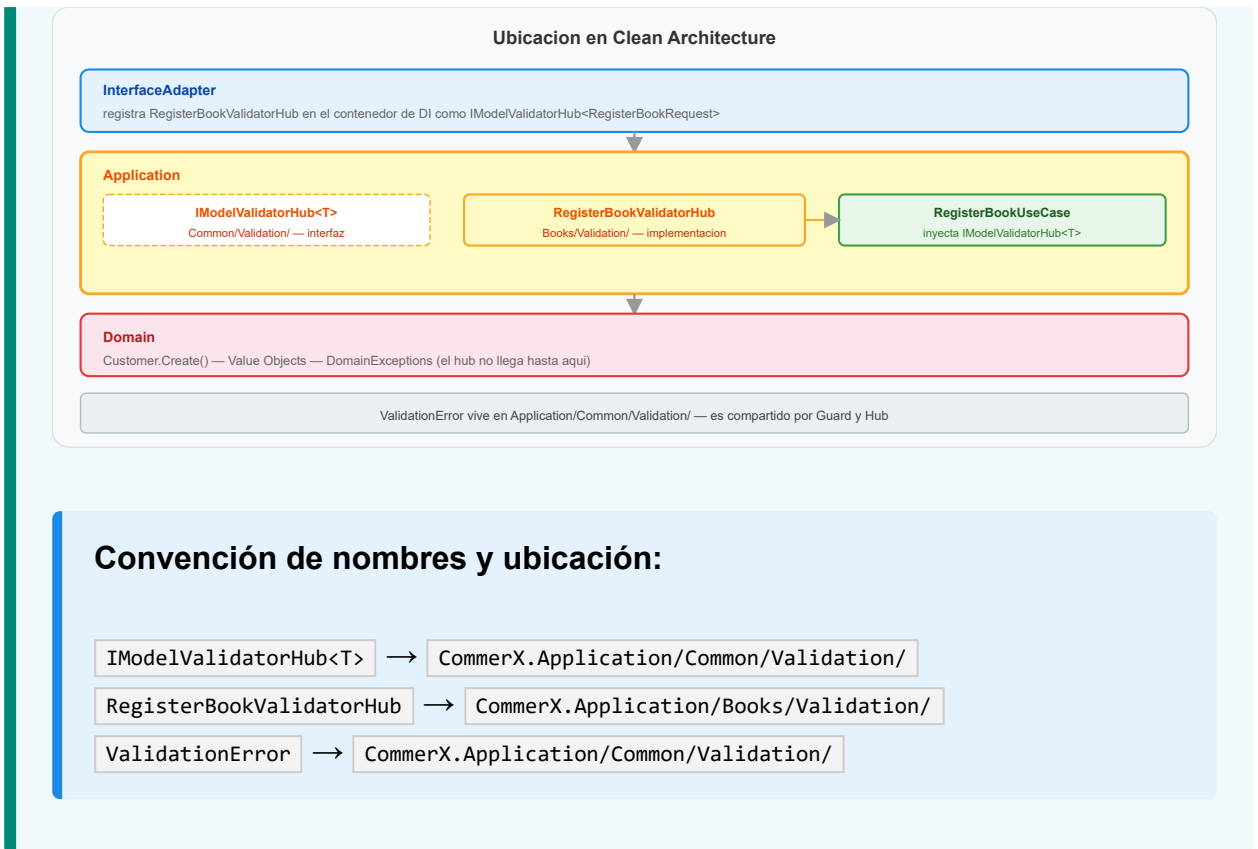
### Tres beneficios concretos del hub:

- 1. Separación de responsabilidades:** el UseCase orquesta el flujo de negocio, el hub valida la entrada — cada uno cambia por razones distintas.
- 2. Testeabilidad:** el hub puede reemplazarse por un mock en los tests del UseCase. También puede testearse de forma independiente con distintos DTOs.
- 3. Reutilización:** si dos UseCases reciben el mismo tipo de request, pueden compartir el mismo hub sin duplicar lógica.

El hub es también una aplicación del principio **Dependency Inversion**: el UseCase depende de la interfaz `IModelValidatorHub<T>` — no de la implementación concreta. Esto permite sustituirla en tests o cambiar la estrategia de validación sin tocar el UseCase.

## ¿Cómo encaja en la arquitectura?

El hub y su interfaz viven en la capa **Application**. La interfaz se define junto a los puertos del UseCase. La implementación concreta se ubica en el mismo namespace del UseCase o en una carpeta dedicada de validación.



## Marco teórico

### Ejemplo completo ejecutable en Main()

El siguiente ejemplo implementa la interfaz, una implementación concreta y un UseCase simulado — todo en un programa de consola sin dependencias externas. Permite verificar el comportamiento completo del hub.

```

1  // DTO de entrada – sin lógica
2  public class RegisterBookRequest
3  {
4      public required string Title { get; init; }
5      public required string Author { get; init; }
6      public required string ISBN { get; init; }
7  }
8
9  // interfaz del hub
10 public interface IModelValidatorHub<TModel>
11 {
12     IEnumerable<ValidationError> Validate(TModel model);
13 }
14

```

```
15 // implementación concreta del hub para RegisterBookRequest
16 public class RegisterBookValidatorHub
17     : IModelValidatorHub<RegisterBookRequest>
18 {
19     public IEnumerable<ValidationError> Validate(RegisterBookRequest r)
20     {
21         var g1 = Guard.Against(r.Title, "Title").NotNullOrEmpty().MaxLength(20
22         0);
23         var g2 = Guard.Against(r.Author, "Author").NotNullOrEmpty().MinLength(2);
24         var g3 = Guard.Against(r.ISBN, "ISBN").NotNullOrEmpty().MinLength(10);
25         return g1.Errors.Concat(g2.Errors).Concat(g3.Errors);
26     }
27 }
28 // Main() – simula el UseCase usando el hub
29 IModelValidatorHub<RegisterBookRequest> hub = new RegisterBookValidatorHub();
30
31 // request con datos inválidos – Title vacío, ISBN corto
32 var request = new RegisterBookRequest
33 {
34     Title = "",
35     Author = "Borges",
36     ISBN = "123" // muy corto – mínimo 10
37 };
38
39 // el UseCase delega al hub – una línea
40 var errors = hub.Validate(request).ToList();
41
42 if (errors.Count > 0)
43 {
44     foreach (var e in errors)
45         Console.WriteLine($"[{e.PropertyName}] {e.ErrorMessage}");
46     // [Title] El campo 'Title' es requerido.
47     // [ISBN] 'ISBN' debe tener al menos 10 caracteres.
48     return; // no continúa – dominio no es invocado
49 }
50
51 Console.WriteLine("Datos validos – continua el UseCase.");
```

## Errores comunes

### Error 1 — poner lógica de negocio en el hub

El hub solo debe contener Guards — verificaciones de precondiciones técnicas. Verificar unicidad en la base de datos, calcular descuentos o aplicar reglas del dominio son responsabilidades del UseCase o del dominio, nunca del hub.

#### ERROR COMUN

El hub consulta el repositorio — mezcla validación con acceso a datos.

```
1 public IEnumerable<ValidationError> Validate(RegisterBookRequest r)
2 {
3     // MAL: el hub accede al repositorio — no es su responsabilidad
4     var existe = _repo.ExistsByISBN(r.ISBN);
5     if (existe) yield return new ValidationError("ISBN", "Ya existe.");
6 }
```

#### BUENA PRACTICA

El hub solo valida precondiciones. La unicidad la verifica el UseCase después.

```
1 // hub: solo precondiciones técnicas
2 var g = Guard.Against(r.ISBN, "ISBN").NotNullOrEmpty().MinLength(10);
3 return g.Errors;
4 // el UseCase verifica unicidad con _repo.ExistsByISBN() — después del hub
```

### Error 2 — crear un hub genérico para todos los requests

Un único hub que valida cualquier tipo de request conduce a condicionales internos complejos y mezcla las reglas de distintos modelos. Cada request debe tener su propio hub — el nombre lo indica claramente.

**ERROR COMUN**

Hub genérico con condicionales — difícil de mantener y extender.

```
1 // MAL: un hub para todo — mezcla responsabilidades
2 public class GeneralValidatorHub
3 {
4     public IEnumerable<ValidationError> Validate(object request)
5     {
6         if (request is RegisterBookRequest r1) { /* ... */ }
7         if (request is PlaceOrderRequest r2) { /* ... */ }
8     }
9 }
```

**BUENA PRACTICA**

Un hub por tipo de request — cohesivo, fácil de leer y de testear.

```
1 // cada request tiene su hub especializado
2 public class RegisterBookValidatorHub
3     : IModelValidatorHub<RegisterBookRequest> { /* ... */ }
4
5 public class PlaceOrderValidatorHub
6     : IModelValidatorHub<PlaceOrderRequest> { /* ... */ }
```

**Qué NO hacer****No inyectar el hub directamente como clase concreta**

Si el UseCase recibe `RegisterBookValidatorHub` en lugar de `IModelValidatorHub<RegisterBookRequest>`, la dependencia es concreta — no se puede sustituir en tests ni intercambiar la implementación. Siempre se debe depender de la interfaz.

**QUE NO HACER**

El UseCase recibe la clase concreta — acoplamiento directo.

```
1 // MAL: depende de la implementación, no de la interfaz
2 public RegisterBookUseCase(RegisterBookValidatorHub hub)
3     => _validator = hub;
```

**BUENA PRACTICA**

El UseCase recibe la interfaz — el contenedor de DI inyecta la implementación.

```
1 // BIEN: depende de la abstracción
2 public RegisterBookUseCase(
3     IModelValidatorHub<RegisterBookRequest> validator)
4     => _validator = validator;
```

**Resumen — reglas del hub:**

- El hub SOLO contiene Guards — sin lógica de negocio, sin acceso a datos.
- Un hub por tipo de request — nunca un hub genérico para todo.
- El UseCase SIEMPRE depende de la interfaz, no de la clase concreta.
- La unicidad y otras verificaciones con repositorio van en el UseCase, después del hub.